

Đường đi ngắn nhất nguồn đơn trong đồ thị có hướng không chu trình

Lecture Notes #17

Nguồn gốc: Single-Source Shortest Paths in Directed Acyclic Graphs

English Materials / Longest Path on DAG.pdf

1 Đồ thị DAG và thứ tự tô-pô

Trong một đồ thị có hướng không chu trình, gọi tắt là DAG, không tồn tại chu trình. Vì vậy, chu trình âm không thể tồn tại trong DAG, và bài toán đường đi ngắn nhất được định nghĩa tốt.

Bài toán đường đi ngắn nhất nguồn đơn trên DAG có thể giải hiệu quả hơn nhờ sắp xếp tô-pô.

Một thứ tự tô-pô của DAG $G = (V, E)$ là một thứ tự tuyến tính của toàn bộ các đỉnh sao cho nếu G có cạnh (u, v) thì u xuất hiện trước v trong thứ tự. Có thể nhìn thứ tự tô-pô như cách đặt các đỉnh trên một đường ngang sao cho mọi cạnh có hướng đều đi từ trái sang phải.

2 Thuật toán sắp xếp tô-pô

Sắp xếp tô-pô là một ứng dụng của tìm kiếm theo chiều sâu.

Thủ tục TOPSORT(G).

1. Đánh dấu mọi đỉnh $v \in G$ là *chưa thăm*.
2. Khi còn tồn tại một đỉnh v chưa thăm, gọi SORT(G, v).

Thủ tục SORT(G, v).

1. Đánh dấu v là *đã thăm*.
2. In v .
3. Với mỗi $u \in \text{Adj}[v]$, nếu u chưa thăm thì gọi SORT(G, u).

Một cách minh họa trực quan là đặt cùng một DAG theo thứ tự tô-pô, sao cho mọi cạnh đều đi từ trái sang phải. Độ phức tạp thời gian của TOPSORT là

$$O(|V| + |E|).$$

3 Đường đi ngắn nhất trên DAG

Thuật toán sau giải bài toán đường đi ngắn nhất nguồn đơn trên DAG có trọng số.

Thủ tục DAG-SHORTEST-PATHS(G, s, w).

1. Gọi TOPSORT(G).
2. Gọi INITIALIZE-SINGLE-SOURCE(G, s).
3. Đặt $S := \emptyset$.
4. Xét từng đỉnh u theo thứ tự tô-pô. Với mỗi $v \in \text{Adj}[u]$, thực hiện RELAX(u, v, w), rồi đưa u vào S .

Hai thủ tục con giống như trong thuật toán Bellman–Ford.

Thủ tục INITIALIZE-SINGLE-SOURCE(G, s).

1. Với mỗi đỉnh $v \in V$, đặt $d[v] := \infty$ và $\pi[v] := \text{nil}$.
2. Đặt $d[s] := 0$.

Thủ tục RELAX(u, v, w). Nếu

$$d[v] > d[u] + w(u, v),$$

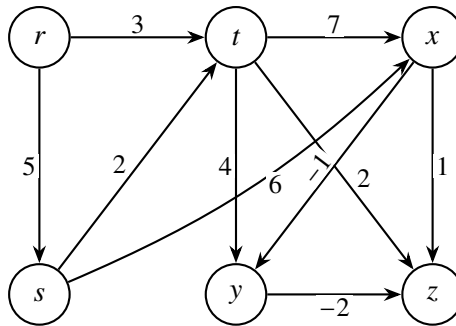
thì cập nhật

$$d[v] := d[u] + w(u, v), \quad \pi[v] := u.$$

Ví dụ trong bản gốc chạy thuật toán trên đồ thị ở Hình 1. Thứ tự tô-pô là

$$r, s, t, x, y, z.$$

Nguồn là s , vì vậy r không đạt được từ nguồn. Thuật toán duyệt các đỉnh theo thứ tự trên và thư giãn mọi cạnh đi ra từ đỉnh đang xét.



Hình 1: DAG ví dụ trong bài giảng; các cạnh đều đi từ trái sang phải theo một thứ tự tô-pô.

Các giá trị cuối cùng của ví dụ là:

v	r	s	t	x	y	z
$d[v]$	∞	0	2	6	5	3
$\pi[v]$	nil	nil	s	s	x	y

Chẳng hạn, đường đi ngắn nhất tới z là

$$s \rightarrow x \rightarrow y \rightarrow z$$

có trọng số $6 + (-1) + (-2) = 3$.

4 Tính đúng đắn

Định nghĩa $\delta(s, v)$ như thường lệ:

$$\delta(s, v) = \begin{cases} \min\{w(p) : s \xrightarrow{P} v\}, & \text{nếu tồn tại đường đi từ } s \text{ đến } v, \\ \infty, & \text{ngược lại.} \end{cases}$$

Cho một đường đi

$$P = v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_m.$$

Các đỉnh trong $\{v_1, v_2, \dots, v_{j-1}\}$ được gọi là các tiền nhiệm của v_j trên P .

Gọi một đường đi ngắn nhất P từ s đến v sao cho mọi tiền nhiệm của v đều nằm trong S là *đường đi ngắn nhất từ s đến v tương ứng với S* .

Bổ đề 1. Gọi

$$(u_1, u_2, \dots, u_n)$$

là danh sách đỉnh theo thứ tự tô-pô, với $n = |V|$ và $u_i = s$. Ngay sau vòng lặp ngoài thứ j của DAG-SHORTEST-PATHS, với mọi $k > j$, giá trị $d[u_k]$ là trọng số đường đi ngắn nhất từ s đến u_k tương ứng với S . Hơn nữa,

$$d[u_{j+1}] = \delta(s, u_{j+1}).$$

Chứng minh. *Cơ sở:* $j = i$. Sau vòng lặp thứ i , $S = \{s\}$ và $d[u_k] = w(s, u_k)$; bổ đề đúng.

Giả thiết quy nạp: giả sử bổ đề đúng với $j = m < n - 1$.

Bước quy nạp: xét $j = m + 1$. Ở vòng lặp thứ $m + 1$, đỉnh u_{m+1} được đưa vào S , và $\text{RELAX}(u_{m+1}, v, w)$ được gọi cho mọi $v \in \text{Adj}[u_{m+1}]$. Do thứ tự tô-pô, các đỉnh này đều nằm bên phải u_{m+1} .

Nếu u_{m+1} không đạt được từ s , thì mọi v được xét ở đây cũng không đạt được từ s qua u_{m+1} , và giá trị $d[v]$ vẫn là ∞ khi không có đường khác.

Nếu u_{m+1} đạt được từ s , thì với cạnh (u_{m+1}, v) , phép thử giãn kiểm tra liệu đường đi qua u_{m+1} có tốt hơn đường đi ngắn nhất hiện tại tương ứng với tập tiền nhiệm cũ hay không. Khi

$$d[v] > d[u_{m+1}] + w(u_{m+1}, v),$$

ta cập nhật

$$d[v] := d[u_{m+1}] + w(u_{m+1}, v).$$

Theo giả thiết quy nạp, giá trị mới này là trọng số của đường đi ngắn nhất từ s đến v tương ứng với tập mới

$$S = \{u_i, u_{i+1}, \dots, u_{m+1}\}.$$

Ngoài ra, nếu $v = u_{m+2}$ thì không có đỉnh nào trong

$$\{u_{m+3}, u_{m+4}, \dots, u_n\}$$

có thể đi đến u_{m+2} , vì thứ tự tô-pô đặt mọi cạnh từ trái sang phải. Do đó

$$d[v] = \delta(s, v).$$

Bổ đề đúng cho vòng lặp thứ $m + 1$, hoàn tất quy nạp.

Định lý 1. Thuật toán DAG-SHORTEST-PATHS, chạy trên DAG có trọng số $G = (V, E)$ với nguồn s , kết thúc với

$$d[v] = \delta(s, v)$$

cho mọi đỉnh $v \in V$.

Chứng minh. Theo Bổ đề 1, sau vòng lặp thứ j , giá trị

$$d[u_{j+1}] = \delta(s, u_{j+1})$$

đã được chốt. Sau $n - 1$ vòng lặp, mọi đỉnh đều có giá trị đúng. Vòng lặp ngoài thật ra chạy n lần; vòng cuối cùng là dư thừa nhưng không làm thay đổi tính đúng đắn.

5 Phân tích độ phức tạp

- TOPSORT tốn $O(|V| + |E|)$ thời gian.
- INITIALIZE-SINGLE-SOURCE tốn $O(|V|)$ thời gian.
- Vòng lặp ngoài chạy $|V|$ lần. Tuy vậy, xét tổng toàn bộ vòng lặp trong, mỗi cạnh chỉ được duyệt đúng một lần từ trái sang phải, nên tổng số lần lặp bên trong là $|E|$. Mỗi lần lặp bên trong tốn $O(1)$.

Vì vậy tổng thời gian chạy là

$$O(|V| + |E|).$$

6 Ghi chú về đường đi dài nhất

Với DAG, bài toán đường đi dài nhất cũng có thể xử lý bằng cùng ý tưởng thứ tự tô-pô, vì không có chu trình để làm đường đi tăng vô hạn. Một cách nhìn đơn giản là đổi dấu trọng số cạnh rồi chạy thuật toán đường đi ngắn nhất trên DAG; hoặc trực tiếp thay phép thư giãn nhỏ nhất bằng phép thư giãn lớn nhất. Thứ tự tô-pô vẫn bảo đảm khi xét một đỉnh, mọi tiền nhiệm của nó đã được xử lý.